

Confidential

Application Note

Interface Specification for Touch Screen Controller

**2023. 11. 01
Rev. 1.1.0**

[HISTORY]

Rev.	Date	Author	Description
1.1.0	2023 / 11 / 01	Hyun Jun Park	11 th Draft - Added ZTL760 IC
1.0.9	2023 / 10 / 19	Hyun Jun Park	10 th Draft - Modified Read/Write Data I2c Packet Diagram
1.0.8	2023 / 10 / 12	Hyun Jun Park	9 th Draft - Add Read/Write Data I2c Packet Diagram
1.0.7	2022 / 06 / 03	Hyun Jun Park	8 th Draft - Fix indicator for chapter 5 and above
1.0.6	2022 / 01 / 12	Oh young kwon	7 th Draft - Update IC list.(GTW623 , ZTM730)
1.0.5	2020 / 05 / 25	Un Hyuk Kwak	6 th Draft - Added decrption for how to read raw data. (Chapter 5. For Reading Raw Data) - Update "Chapter 2"
1.0.4	2020 / 04 / 16	Un Hyuk Kwak	5 th Draft - Added delay after command (Chapter 3. Firmware Upgrade) - Delete for not using function. (Chapter 4.2)
1.0.3	2019 / 12 / 19	Hee Dong Seo	4 th Draft - Delete for not using function. (Chapter 4)
1.0.2	2019 / 11 / 20	Hee Dong Seo	3 rd Draft - Added Panel Test. (Chapter 7) - Added Function (Command) list. (Chapter 8) - Update IC list. (ZT2717 / ZTW622 / FZ01) - Update "Chapter 2". (Added sample code) - Update "Chapter 4". (Added sample code) - Update "Chapter 6".
1.0.1	2019 / 09 / 09	Hee Dong Seo	2 nd Draft - Added delay after command [FWW_CMD01] (Chapter 3. Firmware Upgrade)
1.0.0	2019 / 08 / 30	Hee Dong Seo	1 st Draft

[INDEX]

1. I2C Packet Protocol.....	4
1.1. I2C Device Address & Define Terms	4
1.2. Write Packet	4
1.3. Read Packet	4
1.4. I2C Handshaking	5
2. Power Sequence	6
2.1. Booting Command Function	7
2.2. Saving TSM Module ID Function (JIG Option)	7
3. Firmware Upgrade	8
4. For Reading Point & Button Information	9
4.1. Point & Button Info Packet	9
4.2. Point & Button Info Structure	10
4.3. How to parsing point information	11
5. For Reading Raw Data.....	13
6. IC Command List.....	14
7. IC Registers.....	15
7.1. Information Register	15
7.2. Sensitivity Register	15
7.3. Common Setting Register	16
7.4. Reporting Register	17
8. Panel Test	18
8.1. DND	18
8.2. DND V gap / DND H gap	18
8.3. HF DND	18
8.4. HF DND V gap / HF DND H gap.....	18
8.5. Self DND	18
8.6. Self DND gap	18
8.7. Self Saturation TRX	18
8.8. TRX Short	18
8.9. Calculate gap data	19
9. Function (Command) list	20
[Appendix].....	21

1. I2C Packet Protocol

1.1. I2C Device Address & Define Terms

- **Slave Address Value** : 0x20 (@7bit)
- **START** : I2C Start Condition
- **STOP** : I2C Stop Condition
- **SAW** : Slave Address Write (0x40)
- **SAR** : Slave Address Read (0x41)
- **CMD** : Command (2byte)
- **RA** : Register Address (2byte)
- **WRD** : Write Register Data (2byte)
- **RRD** : Read Register Data (2byte)

※ **alpha group IC** : ZT7554, ZT7548, ZT7538, ZT2628, ZTW522, ZT2628, ZTW523, ZTM620, ZT2717, ZTW622, FZ01, GTW623, ZTM730

※ **beta group IC** : ZT7650, ZT7650M, ZTL650, ZTL760

※ The alpha group ICs are not support "repeated start"

※ The slave address of ZTL760 IC is 0x40(@7bit)

1.2. Write Packet

[Write Command]

START	SAW	ACK	CMD[7:0]	ACK	CMD[15:8]	ACK	STOP
-------	-----	-----	----------	-----	-----------	-----	------

After transmitting Write Slave Address, transmit 2byte Command and then generate STOP condition.

[Write Register Value]

START	SAW	ACK	RA[7:0]	ACK	RA[15:8]	ACK	WRD[7:0]	ACK	WRD[15:8]	ACK	STOP
-------	-----	-----	---------	-----	----------	-----	----------	-----	-----------	-----	------

After transmitting Write Slave Address, transmit 2byte Register Address and transmit 2byte Register Value subsequently. After this, generate STOP condition.

[Write Data]

START	SAW	ACK	RA[7:0]	ACK	RA[15:8]	ACK	DATA1[7:0]	ACK	DATA1[15:8]	ACK
	DATA2[7:0]	ACK	DATA2[15:8]	...	DATA(n)[7:0]	ACK	DATA(n)[15:8]	ACK	STOP	

1.3. Read Packet

After delaying 50us at least, transmitting Read Slave Address and read Data. Then you can acquire the Register Value which you want to read. Generate STOP condition lastly.

[Read Register Value]

START	SAW	ACK	RA[7:0]	ACK	RA[15:8]	ACK	STOP
-------	-----	-----	---------	-----	----------	-----	------

START	SAR	ACK	RRD[7:0]	ACK	RRD[15:8]	NACK	STOP
-------	-----	-----	----------	-----	-----------	------	------

After transmitting Write Slave Address, transmit Address 2byte of Register which you want to read and then generate STOP condition.

[Read Data]

START	SAW	ACK	RA[7:0]	ACK	RA[15:8]	ACK	STOP
-------	-----	-----	---------	-----	----------	-----	------

START	SAR	ACK	DATA1[7:0]	ACK	DATA1[15:8]	...	DATA(n)[7:0]	ACK	DATA(n)[15:8]	NACK	STOP
-------	-----	-----	------------	-----	-------------	-----	--------------	-----	---------------	------	------

1.4. I2C Handshaking

7 TRANSFERRING DATA

7.1 Byte format

Every byte put on the SDA line must be 8-bits long. The number of bytes that can be transmitted per transfer is unrestricted. Each byte has to be followed by an acknowledge bit. Data is transferred with the most significant bit (MSB) first (see Fig.6). If a slave can't receive or transmit another complete byte of data until it has performed some other function, for example servicing an internal interrupt, it can hold the clock line SCL LOW to force the master into a wait state. Data transfer then continues when the slave is ready for another byte of data and releases clock line SCL.

In some cases, it's permitted to use a different format from the I²C-bus format (for CBUS compatible devices for example). A message which starts with such an address can be terminated by generation of a STOP condition, even during the transmission of a byte. In this case, no acknowledge is generated (see Section 10.1.3).

7.2 Acknowledge

Data transfer with acknowledge is obligatory. The acknowledge-related clock pulse is generated by the master. The transmitter releases the SDA line (HIGH) during the acknowledge clock pulse.

The receiver must pull down the SDA line during the acknowledge clock pulse so that it remains stable LOW

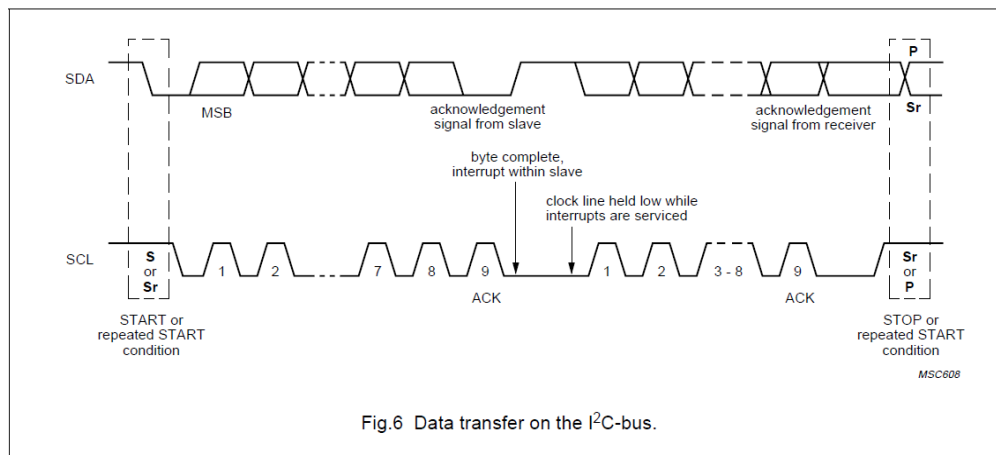
during the HIGH period of this clock pulse (see Fig.7). Of course, set-up and hold times (specified in Section 15) must also be taken into account.

Usually, a receiver which has been addressed is obliged to generate an acknowledge after each byte has been received, except when the message starts with a CBUS address (see Section 10.1.3).

When a slave doesn't acknowledge the slave address (for example, it's unable to receive or transmit because it's performing some real-time function), the data line must be left HIGH by the slave. The master can then generate either a STOP condition to abort the transfer, or a repeated START condition to start a new transfer.

If a slave-receiver does acknowledge the slave address but, some time later in the transfer cannot receive any more data bytes, the master must again abort the transfer. This is indicated by the slave generating the not-acknowledge on the first byte to follow. The slave leaves the data line HIGH and the master generates a STOP or a repeated START condition.

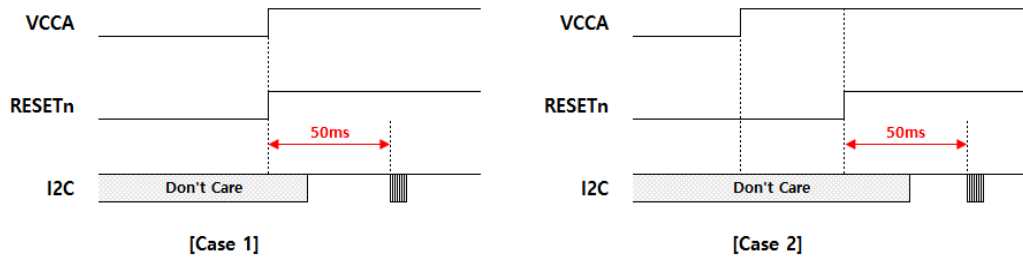
If a master-receiver is involved in a transfer, it must signal the end of data to the slave- transmitter by not generating an acknowledge on the last byte that was clocked out of the slave. The slave-transmitter must release the data line to allow the master to generate a STOP or repeated START condition.



S	DA[7:1]	0	ACK with SCL=0 (I2C SLAVE holding)	RA[7:0]	A	DATA : 1st BYTE	A	P
S	DA[7:1]	1	ACK with SCL=0 (I2C SLAVE holding)	DATA : 1st BYTE	A		N	P

HOST should check "SCL" after generating clock to get ACK from I2C SLAVE. I2C SLAVE will hold SCL=0 with ACK after receiving DEVICE ADDRESS, until it finish the task which was received/transmitted previously.

2. Power Sequence



You need first step (Power sequence) for touch IC running after power on.

```

u16 ret, value;

power_control(Power_OFF);
//50ms Delay : The time depends on the Power CAP size
power_control(Power_ON);
//50ms Delay : The time depends on the Power CAP size

//Read checksum result value to distinguish whether AUTO_BOOT is or not.
ret = read(0x012C, (u8 *)&value); //Reading Checksum, Refer to "Chapter 7.1"

//Firmware is broken or empty or NO AUTO_BOOT
IF (value != 0x55AA) {
    run "Bootling Command Function" <----- Refer to "Chapter 2.1"
    ret = read_reg(0x012C, (u8 *)&value); //Reading Checksum, Refer to "Chapter 7.1"
    IF (value != 0x55AA) {ERROR!!!, Do firmware upgrade} //Firmware is broken or empty
}

//IC Bootling is OK and then read version information
write_reg(0x000A, 0x000A); //A-sync I2C Start command, Refer to "Chapter 6"
read_reg(0x0011, (u8 *)&ic_name);
read_reg(0x001C, (u8 *)&ic_vendor_id);
read_reg(0x0014, (u8 *)&hw_id);
read_reg(0x001E, (u8 *)&tsm_module_id);
read_reg(0x0012, (u8 *)&major_fw_version);
read_reg(0x0121, (u8 *)&minor_fw_version);
read_reg(0x0013, (u8 *)&release_version);
IF (Checking version == Fail) {ERROR!!!, Do firmware upgrade}

write_cmd(0x0003); //Clear Interrupt pin (Go to High), Refer to "Chapter 6"
write_cmd(0x0003); //Clear Interrupt pin (Go to High), Refer to "Chapter 6"
write_cmd(0x0003); //Clear Interrupt pin (Go to High), Refer to "Chapter 6"

//[JIG Option] If no need to saving "module ID", do not run this step.
write_reg(0x000A, 0x000A); //A-sync I2C Start command, Refer to "Chapter 6"
read_reg(0x001E, (u8 *)&tsm_module_id); //Read module ID
IF (tsm_module_id is not matched) {run "Saving TSM Module ID Function"} <----- Refer to "Chapter 2.2"

```

2.1. Booting Command Function

```
u16 chip_code;

write_reg(CMD01, 0x0001); //Refer to "Table.1"
write_cmd(CMD03); //Refer to "Table.1"
read_reg(CMD02, (u8 *)&chip_code); //Refer to "Table.1"
printf("chip code = 0x%x\n", chip_code); //Refer to "Appendix"

write_cmd(CMD03); //Refer to "Table.1"
write_reg(CMD04, 0x0001); //Refer to "Table.1"
//2ms Delay <----- Mandatory

write_reg(CMD05, 0x0001);
//50ms Delay <----- Mandatory
```

CMD	Alpha Group	Beta Group
CMD01	0xC000	0x10F0
CMD02	0xCC00	0x17F0
CMD03	0xC004	0x14F0
CMD04	0xC002	0x12F0
CMD05	0xC001	0x11F0
CMD06	0xC003	0x13F0

[Table.1] Vendor Command

2.2. Saving TSM Module ID Function (JIG Option)

```
u16 ret, value;

write_reg(0x000A, 0x000A); //A-sync I2C Start command, Refer to "Chapter 6"
write_reg(0x001E, TSM_Module_ID); //"TSM_Module_ID" Value is defined by user, Refer to "Chapter 7.1"

write_reg(CMD06, 0x0001); //Refer to "Table.1"
//10ms Delay <----- Mandatory
write_cmd(0x0007); //Save register command, Refer to "Chapter 6"
//1000ms Delay <----- Mandatory

write_reg(0x000A, 0x000A); //A-sync I2C Start command, Refer to "Chapter 6"
write_reg(CMD06, 0x0000); //Refer to "Table.1"
//10ms Delay <----- Mandatory

RE-START >>>>>>
```

Command	Alpha Group	Beta Group
FWW_CMD01	0x01D5	0x25F0
FWW_CMD02	0xC000	0x10F0
FWW_CMD03	0xC004	0x14F0
FWW_CMD04	0xCC00	0x17F0
FWW_CMD05	0xC002	0x12F0
FWW_CMD06	0xC003	0x13F0
FWW_CMD07	0x01DF	0x28F0
FWW_CMD08	0x01D0	0x20F0
FWW_CMD09	0x01DE	0x27F0
FWW_CMD10	0x01D1	0x21F0
FWW_CMD11	0x01D2	0x22F0

[illegible]

4. For Reading Point & Button Information

```

u16 Status_Register;
u16 Finger_Cnt_Register

write_reg(0x000A, 0x000A); //A-sync I2C Start command, Refer to "Chapter 6"

write_cmd(0x0009);  <----- [JIG option] JIG Mode On Command, Refer to "Chapter 6"
//100ms Delay

write_reg(0x0010, 0x0000); <----- Mandatory (Setting to Touch Mode = 0, Normal Working Mode)

write_cmd(0x0006);  <----- [JIG option] SW Cal. Command, Refer to "Chapter 6"
//300ms Delay

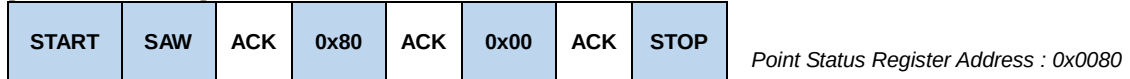
write_cmd(0x0003); <----- Mandatory (Clear Interrupt pin : Go to High)

Loop {
    Waiting INT pin active low
    Read "Point Information"
    Read "Button Information" <----- Option
    write_cmd(0x0003);
    Parsing the information and send the events
}

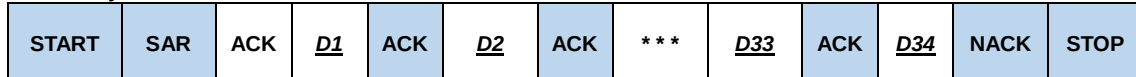
```

4.1. Point & Button Info Packet

[Point Info Packet]



50us Delay

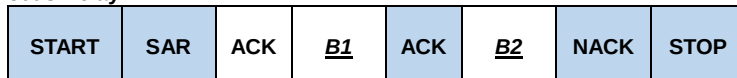


※ D1 ~ D34 : Point Information (Refer to "Chapter 4.2 - Point Data")

[Button Info Packet]



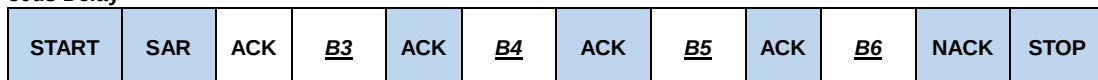
50us Delay



10us Delay



50us Delay



※ B1 ~ B6 : Button Information (Refer to "Chapter 4.2 - Button Data")

4.2. Point & Button Info Structure

[Point Data]

Info	Data		Register Name	Description
Information	D1	LSB	Status Register	Status1
	D2	MSB		
	D3	LSB	Finger Num(Time Stamp)	Status2
	D4	MSB		
Finger 0	D5	LSB	Point A0	X Coord
	D6	MSB		
	D7	LSB	Point A1	Y Coord
	D8	MSB		
	D9	LSB	Point A2	Z Value
	D10	MSB		Sub-status
∫	∫			∫
Finger 4	D29	LSB	Point A12	X Coord
	D30	MSB		
	D31	LSB	Point A13	Y Coord
	D32	MSB		
	D33	LSB	Point A14	Z Value
	D34	MSB		Sub-status

Status Register

BIT 00 : Point count change
 BIT 01 : Point down
 BIT 02 : Point move
 BIT 03 : Point up
 BIT 04 : RESERVED04
 BIT 05 : Palm report gesture
 BIT 06 : Wake up
 BIT 07 : RESERVED07
 BIT 08 : RESERVED08
 BIT 09 : RESERVED09
 BIT 10 : RESERVED10
 BIT 11 : Point detected
 BIT 12 : RESERVED12
 BIT 13 : MUST ZERO
 BIT 14 : Debug
 BIT 15 : Button info

Sub-Status

BIT 00 : Sub point exist
 BIT 01 : Sub point down
 BIT 02 : Sub point move
 BIT 03 : Sub point up
 BIT 04 : RESERVED04
 BIT 05 : RESERVED05
 BIT 06 : RESERVED06
 BIT 07 : Sub palm

[Button Data]

Info	Data		Register Name	Description
Status	B1	LSB	Button Status Register	Button Up or Down
	B2	MSB		
Width	B3	LSB	Button Width 1	Strength of Button 1
	B4	MSB		
	B5	LSB	Button Width 2	Strength of Button 2
	B6	MSB		

Button Status Register

BIT 00 : BUTTON 0 DOWN
 BIT 01 : BUTTON 1 DOWN
 BIT 02 : BUTTON 2 DOWN
 BIT 03 : BUTTON 3 DOWN
 BIT 04 : BUTTON 4 DOWN
 BIT 05 : BUTTON 5 DOWN
 BIT 06 : BUTTON 6 DOWN
 BIT 07 : BUTTON 7 DOWN

BIT 08 : BUTTON 0 UP
 BIT 09 : BUTTON 1 UP
 BIT 10 : BUTTON 2 UP
 BIT 11 : BUTTON 3 UP
 BIT 12 : BUTTON 4 UP
 BIT 13 : BUTTON 5 UP
 BIT 14 : BUTTON 6 UP
 BIT 15 : BUTTON 7 UP

4.3. How to parsing point information

Define “ Point Info Struct & Variable, Util ”

```

Typedef struct
{
    u16    x;
    u16    y;
    u8     width;
    u8     sub_status;
}zinitix_coord;

Typedef struct
{
    u16          status;
    u8           finger_cnt;
    u8           time_stamp;    //(Option)
    ts_zinitix_coord coord[5];  //Support finger Num.
}zinitix_point_info;

zinitix_point_info    current_touch_info;
zinitix_point_info    previous_touch_info;

#define zinitix_bit_test( val, n )  (((val)>>(n)) & (0x01))
  
```

Sample Code “ Interrupt Handler ”

```
static void IRQHANDLE Touch_Work (void)
{
    read_data( 0x0080, (u8 *) &touch_info, sizeof(zinitix_point_info) );
    write_cmd( 0x0003 );

    Report_Touch_Event();
    return;
}
```

INT Pin of Touch IC is connected to GPIO of AP.

So, Interrupt processing should be done to enable AP to recognize GPIO as Low.

(Interrupt Type: Falling Edge & Low Level)

Sample Code “ Parsing point data ”

```
void Report_Touch_Event(void)
{
    if( !zinitix_bit_test( current_touch_info.status, 11 ) ) { //No Points State
        for( i = 0; i < 5; i++ ) {
            if( zinitix_bit_test( previous_touch_info.coord[i].sub_status, 0 ) ) { //Check for exist the each point
                REPORT_TO_HOST( previous_touch_info.coord[i].x, previous_touch_info.coord[i].y,
                    previous_touch_info.coord[i].width, FINGER_UP);
            }
        }
        Memset ( &previous_touch_info, 0x00, sizeof(zinitix_point_info) );
    }
    else {
        for( i = 0; i < 5; i++ ) {
            if( zinitix_bit_test( current_touch_info.coord[i].sub_status, 1 ) ) {
                REPORT_TO_HOST( current_touch_info.coord[i].x, current_touch_info.coord[i].y,
                    current_touch_info.coord[i].width, FINGER_DOWN);
            }
            else if( zinitix_bit_test( current_touch_info.coord[i].sub_status, 2 ) ) {
                REPORT_TO_HOST( current_touch_info.coord[i].x, current_touch_info.coord[i].y,
                    current_touch_info.coord[i].width, FINGER_MOVE);
            }
            else if( zinitix_bit_test( current_touch_info.coord[i].sub_status, 3 ) )
            {
                REPORT_TO_HOST( previous_touch_info.coord[i].x, previous_touch_info.coord[i].y,
                    previous_touch_info.coord[i].width, FINGER_UP);
                Memset ( &current_touch_info.coord[i] , 0x00, sizeof(zinitix_coord) );
            }
            else if( zinitix_bit_test( current_touch_info.coord[i].sub_status, 0 ) ) { //Do nothing
            }
            else { Memset ( &current_touch_info.coord[i] , 0x00, sizeof(zinitix_coord) ); }
        }
       Memcpy ( &previous_touch_info, &current_touch_info, sizeof(zinitix_point_info) );
    }
    return;
}
```

5. For Reading Raw Data

Raw Data is used to acquire characteristic data or sensitivity data for each Node. There are different types of data, so you need to set the data operation mode which you would like to acquire first in order to read the normal value.

- The data size of 1Node : 2Byte
- Total Node : (Number of TX Channel) x (Number of RX Channel)
- Setting operation mode : Write operation mode value to TOUCH MODE Register. (Refer to “Chapter 7.3”)

[Example]

If TX Channel = 10 / RX Channel = 5, Total Node = 10x5 = 50.

Need to read 100byte for acquiring Raw Data of 50 Node, acquisition data are N0, N1, N2, N3, , N47, N48, N49. The acquisition Raw data is aligned with the channel layout as following table.

	Y0	Y1	Y2	Y3	Y4
X0	N00	N01	N02	N03	N04
X1	N05	N06	N07	N08	N09
X2	N10	N11	N12	N13	N14
X3	N15	N16	N17	N18	N19
X4	N20	N21	N22	N23	N24
X5	N25	N26	N27	N28	N29
X6	N30	N31	N32	N33	N34
X7	N35	N36	N37	N38	N39
X8	N40	N41	N42	N43	N44
X9	N45	N46	N47	N48	N49

```

write_reg(0x0010, XXXX); //Set "touch mode" , Refer to "Chapter 8"
//100ms Delay
write_cmd(0x0003); <----- Mandatory (Clear Interrupt pin : Go to High)

Waiting INT pin active low and then write_cmd(0x0003); <---- Mandatory (Clear Interrupt pin : Go to High)
Waiting INT pin active low and then write_cmd(0x0003); <---- Mandatory (Clear Interrupt pin : Go to High)
Waiting INT pin active low and then write_cmd(0x0003); <---- Mandatory (Clear Interrupt pin : Go to High)
Waiting INT pin active low and then write_cmd(0x0003); <---- Mandatory (Clear Interrupt pin : Go to High)

Waiting INT pin active low
Read_Data(0x0200, &Data, N) // N : Total Node Number
write_cmd(0x0003);
write_cmd(0x0003);

write_reg(0x0010, 0x0000); //Set "touch mode = Normal Mode"
write_cmd(0x0003);

```

[Read Data (Burst Read)]

START	SAW	ACK	0x00	ACK	0x20	ACK	STOP
-------	-----	-----	------	-----	------	-----	------

1ms Delay

START	SAR	ACK	DATA1 [7:0]	ACK	DATA1 [15:8]	..	DATA(n) [7:0]	ACK	DATA(n) [15:8]	NACK	STOP
-------	-----	-----	----------------	-----	-----------------	----	------------------	-----	-------------------	------	------

6. IC Command List

Command	Name	DESCRIPTION
0x0000	SW Reset CMD	Reset software
0x0001	Wake Up CMD	Go to active mode
0x0002	NVM Auto LPM Off CMD	Disable the "NVM AUTO LPM"
0x0003	Clear Interrupt CMD	Making INT pin to high voltage
0x0004	Deep Sleep CMD	Go to deep sleep mode
0x0005	Sleep CMD	Go to sleep mode (Waiting for gesture input)
0x0006	SW Calibration CMD	Reset baseline (Reset Ref. data)
0x0007	Save Register CMD	Saving current register value into the IC
0x0008	Save HW Calibration CMD	Saving HW Calibration data into the IC
0x0009	JIG Mode CMD	Using JIG (JIG Mode On)
0x000A	A-sync I2C Start CMD	Notify to the start of I2C
0x000B	A-sync I2C End CMD	Notify to the end of I2C
0x000C	User CMD	T.B.D
0x000D	Exit HID Mode CMD	Exit HID Protocol
0x000E	HID Descriptor CMD	Send HID descriptor
0x000F	Reserved	Reserved

7. IC Registers

7.1. Information Register

RA	REGISTER NAME	RW	Default Val.	DESCRIPTION
0x0120	CHIP CODE	R	N/A	Chip Code, Refer to "Appendix"
0x0011	IC Name	R	N/A	IC Name
0x001C	IC VENDOR ID	R	0x5A49	IC VENDOR ID : "ZI" (ASCII Code)
0x0012	MAJOR FIRMWARE VERSION	R	N/A	Major Firmware Version
0x0121	MINOR FIRMWARE VERSION	R	N/A	Minor firmware version
0x0013	RELEASE VERSION	R	N/A	Release Version
0x0014	HW ID	RW	N/A	HW ID
0x001E	TSM MODULE ID	RW	N/A	TSM Module ID
0x0018	EEPROM INFO	RW	0xFFFF	Not Using
0x012C	CHECKSUM RESULT	R	0x55AA	Checksum Result (OK : 0x55AA)
0x012E	SYSTEM CHECKSUM_L	R	N/A	Checksum
0x03DF	MNG FILE CHECKSUM	R	N/A	Checksum

7.2. Sensitivity Register

RA	REGISTER NAME	RW	Default Val.	DESCRIPTION
0x001D	CURRENT SENSITIVITY TH	R	N/A	Check Current Sensitivity Threshold
0x0020	SCR SENSITIVITY TH	RW	T.B.D	Sensitivity Threshold (0 ~ 32767)
0x0314	SCR SUM THD	RW	T.B.D	Touched Node Sum THD (0 ~ 32767)
0x003D	SCR SENSITIVITY OFF RATIO	RW	80	Sensitivity Coef. (Unit : %)
0x003B	SCR REACTION CNT	RW	2	Reaction count
0x0134	SX LPI SENSITIVITY	RW	T.B.D	Self X THD
0x0138	SY LPI SENSITIVITY	RW	T.B.D	Self Y THD

SCR SENSITIVITY TH : The reference value used to find the touched area size

SCR SUM THD : Touch On THD

SCR SENSITIVITY OFF RATIO : Touch Off THD (← Touch On THD x Sensitivity Off Ratio)

SCR REACTION CNT : Frame count for changing to touch-on state.

7.3. Common Setting Register

RA	REGISTER NAME	RW	Default Val.	DESCRIPTION
0x0010	TOUCH MODE	RW	0	Set/Load the current touch mode
0x0019	INITIAL TOUCH MODE	RW	0	Set/Load the initial touch mode
0x0015	MAX FINGER NUM	RW	5	Supported Max Finger Num. is 10
0x00C2	COORD ORIENTATION	RW	0x0000	1: X FLIP 2: Y FLIP 4: XY SWAP
0x00C0	X RESOLUTION	RW	T.B.D	LCD X Resolution
0x00C1	Y RESOLUTION	RW	T.B.D	LCD Y Resolution
0x007E	POWER STATE FLAG	RW	0x0000	0: Auto 1: Active 2: Idle
0x00D4	GOTO IDLE TERM	RW	700	After 700ms, Go to idle mode
0x02D4	CFG USE SELF XY	RW	0x0003	0 : Disable X&Y SELF 1 : Only enable X SELF 2 : Only enable Y SELF 3 : Enable X&Y SELF
0x00C1	PIXEL PER MM	RW	T.B.D	LCD DPI (Pixel/mm)

TOUCH MODE : Select operation mode

Kind of operation mode : 0, 20, 11, 12, 13, 17, 39

[00] Normal working mode

[20] Debugging mode – Transmit sensitivity (delta) data

[11] Inspection mode – Transmit PP DND data

(Refer to "[User Guide] ZINITIX_REPORT TSP TEST Item_20191023.pdf")

[12] Inspection mode – Transmit HF DND data

(Refer to "[User Guide] ZINITIX_REPORT TSP TEST Item_20191023.pdf")

[13] Inspection mode – Transmit TRX Short data

(Refer to "[User Guide] ZINITIX_REPORT TSP TEST Item_20191023.pdf")

[17] Inspection mode – Transmit Self DND data

(Refer to "[User Guide] ZINITIX_REPORT TSP TEST Item_20191023.pdf")

[39] Inspection mode – Transmit Self Saturation TRX data

(Refer to "[User Guide] ZINITIX_REPORT TSP TEST Item_20191023.pdf")

INITIAL TOUCH MODE : Touch mode value set after power-up.

POWER STATE FLAG :

[0] Automatically perform to operation scenario

[1] Force active mode – Hybrid (Self + Mutual) sensing

[2] Force idle mode(Active mode without finger) – Self sensing

GOTO IDLE TERM : Time to switch from "Active mode" to "Idle mode"

7.4. Reporting Register

RA	REGISTER NAME	RW	Default Val.	DESCRIPTION
0x007F	DELAY RAW FOR HOSR	RW	0	Raw Data Report for Slow Host
0x00F0	INT ENABLE FLAG	RW	0x003F	Select events
0x00F1	ESD INTERRUPT TERM	RW	0	0 : Disable 1~30000 : Interval
0x00F2	ESD INTERRUPT TIMEOUT	RW	0	ESD Int Clear Check after I2c data Read
0x00FF	INTERRUPT TIMEOUT	RW	0	Int Clear Check after I2c data Read
0x0130	REPORTRATE NPA	RW	120	NPA Report rate (Unit : Hz)
0x0133	REPORTRATE NPI MUTUAL	RW	30	NPI Mutual report rate (Unit : Hz)
0x0136	REPORTRATE NPI SELF	RW	120	NPI Self report rate (Unit : Hz)
0x0137	REPORTRATE LPA	RW	60	LPA Report rate (Unit : Hz)
0x0139	REPORTRATE LPI	RW	40	LPI Report rate (Unit : Hz)

INT ENABLE FLAG : There are 8 events

- BIT 00 : POINT COUNT CHANGE EVENT – [0] Single Touch [1] Occur INT signal when touch count changes
- BIT 01 : DOWN EVENT
- BIT 02 : MOVE EVENT
- BIT 03 : UP EVENT
- BIT 04 : Reserved
- BIT 05 : PALM EVENT – [0] Ignore palm touch [1] Occur INT signal when detect palm touch
- BIT 06 : WAKE UP EVENT – [0] Not using gesture mode [1] Occur INT signal when gesture is recognized
- BIT 07 : Reserved
- BIT 08 : Reserved
- BIT 09 : Reserved
- BIT 10 : Reserved
- BIT 11 : POINT DETECTED EVENT – [0] When down/up/move, only occur INT
[1] If touched, continue to occur INT
- BIT 12 : Reserved
- BIT 13 : MUST ZERO – [0] Not using [1] I2C Error detect bit
- BIT 14 : Reserved
- BIT 15 : BUTTON EVENT – [0] There is no button [1] Using touch button

ESD INTERRUPT TERM : Interval of generating INT signal to the host for ESD PING

ESD INTERRUPT TIMEOUT : Time to keep INT signal low

INTERRUPT TIMEOUT : Time to keep INT signal low

(Host recommends sending "Clear Interrupt CMD" after I2C communication is complete)

8. Panel Test

Change to Touch mode [0] after Data acquisition.

8.1. DND

To check DND data, change Touch mode to [11]. It will be come out the number of (X Channel * Y Channel) and need to read it as ((X Channel * Y Channel) * 2) due to 2Byte unit/node.

8.2. DND V gap / DND H gap

After reading DND data, calculate the output Data and then proceed an inspection. (Refer to "Chapter 7.9")

8.3. HF DND

To check HF DND data, change Touch mode to [12]. It will be come out the number of (X Channel * Y Channel) and need to read it as ((X Channel * Y Channel) * 2) due to 2Byte unit/node.

8.4. HF DND V gap / HF DND H gap

After reading HF DND data, calculate the output Data and then proceed an inspection. (Refer to "Chapter 7.9")

8.5. Self DND

To check Self DND data, change Touch mode to [17]. It will be come out the number of (X Channel + Y Channel) and need to read it as ((X Channel + Y Channel) * 2) due to 2Byte unit/node.

8.6. Self DND gap

After reading Self DND data, calculate the output Data and then proceed an inspection. (Refer to "Chapter 7.9")

Need to check the each of Rx / Tx horizontal gap.

8.7. Self Saturation TRX

To check Self Saturation TRX, change Touch mode to [39]. It will be come out the number of (X Channel + Y Channel) and need to read it as ((X Channel + Y Channel) * 2) due to 2Byte unit/node.

8.8. TRX Short

To check TRX short data, change Touch mode to [13]. It will be come out the number of (X Channel + Y Channel) and need to read it as ((X Channel + Y Channel) * 2) due to 2Byte unit/node.

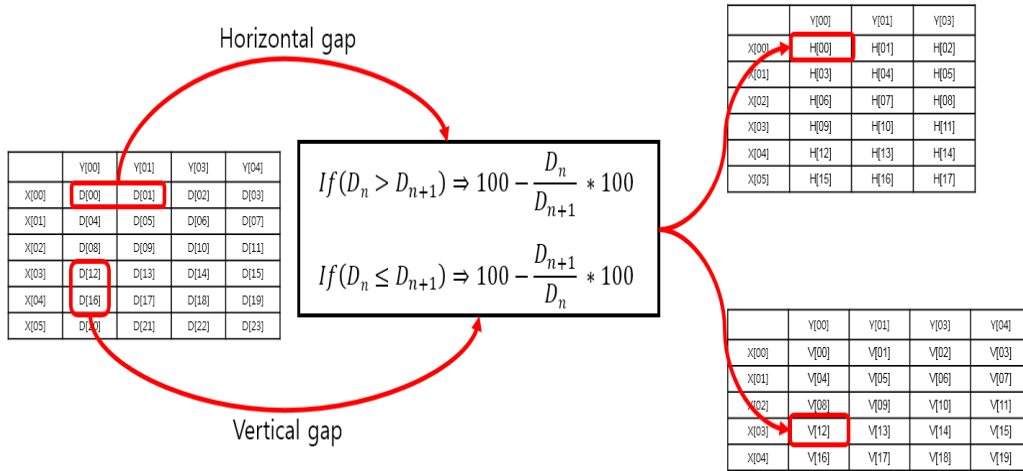
	Normal NODE	Failure NODE
RX	100	5000
TX	200	6000

8.9. Calculate gap data

- The formula for gap is shown below.

$$\text{If}(D_n > D_{n+1}) \Rightarrow 100 - \frac{D_n}{D_{n+1}} * 100$$

$$\text{If}(D_n \leq D_{n+1}) \Rightarrow 100 - \frac{D_{n+1}}{D_n} * 100$$



9. Function (Command) list

Name	Explanation	Usage	Description
Power_control()	Use for Power reset	\$Power_reset [CTRL]	[CTRL]:Power control value 0:Power OFF 1: Power ON
Get_fw_version_ic()	Read version from IC	\$Get_fw_version_ic	-
Get_fw_version_bin()	Read version from Binary	\$Get_fw_version_bin	-
Set_touchmode()	Write Touch mode(0x10)	\$Set_touchmode [MODE]	[MODE]:Touch mode value
Get_touchmode()	Read Touch mode(0x10)	\$Get_touchmode	-
Get_ch_num()	Check channel information	\$Get_ch_num	-
Get_resolution()	Check the resolution	\$Get_resolution	-
Write_reg()	Write a value	\$Write_reg [REG] [VAL]	[REG]:Address value [VAL]:Value
Write_cmd()	Write command	\$Write_cmd [CMD]	[CMD]:Desired command Value (Refer to "Chapter 5")
Read_reg()	Read a value.	\$Read_reg [REG] [NULL]	[REG]:Address value
Touch_work()	Coordinate information output	\$Touch_work	-
Upgrade_firmware()	Firmware upgrade	\$Upgrade_firmware[FW_PATH]	[FW_PATH]: Set the firmware path Ex.) /sdcard/Firmware/fw.bin
Run_dnd_data_read()	dnd data acquisition	\$Run_dnd_data_read	-
Run_dnd_vgap_data_read()	dnd vgap calculation	\$Run_dnd_vgap_data_read	-
Run_dnd_hgap_data_read()	dnd hgap calculation	\$Run_dnd_hgap_data_read	-
Run_hfdnd_data_read()	hfdnd data acquisition	\$Run_hfdnd_data_read	-
Run_hfdnd_vgap_data_read()	hfdnd vgap calculation	\$Run_hfdnd_vgap_data_read	-
Run_hfdnd_hgap_data_read()	hfdnd hgap calculation	\$Run_hfdnd_hgap_data_read	-
Run_selfdnd_data_read()	Self dnd data acquisition	\$Run_selfdnd_data_read	-
Run_selfdnd_gap_data_read()	Self dnd gap calculation	\$Run_selfdnd_gap_data_read	-
Run_self_sat_trx_data_read()	Self saturation trx data acquisition	\$Run_self_sat_trx_data_read	-
Run_trx_short_data_read()	trx short data acquisition	\$Run_trx_short_data_read	-

[Appendix]

IC Name	CHIP Code	NVM Total Size [Byte]	NVM Page Size [Byte]
ZT7528 / ZTW522	0xE532	44*1024	64
ZT7538	0xE538	44*1024	64
ZTL545	0xE545	48*1024	64
ZT7548	0xE548	48*1024	64
ZT2628 / ZTW523 / ZTM620	0xE628	48*1024	64
ZT7650	0xE650	128*1024	1024
ZT2717 / ZTW622 / FZ01	0xE622	48*1024	128
ZT7650M (60pin) / ZTL650	0x650E	96*1024	128
ZT7650M (62pin)	0x650C	96*1024	128
GTW623	0xE623	48*1024	128
ZTM730	0xE730	64*1024	128
ZTL760	0xE760	96*1024	128